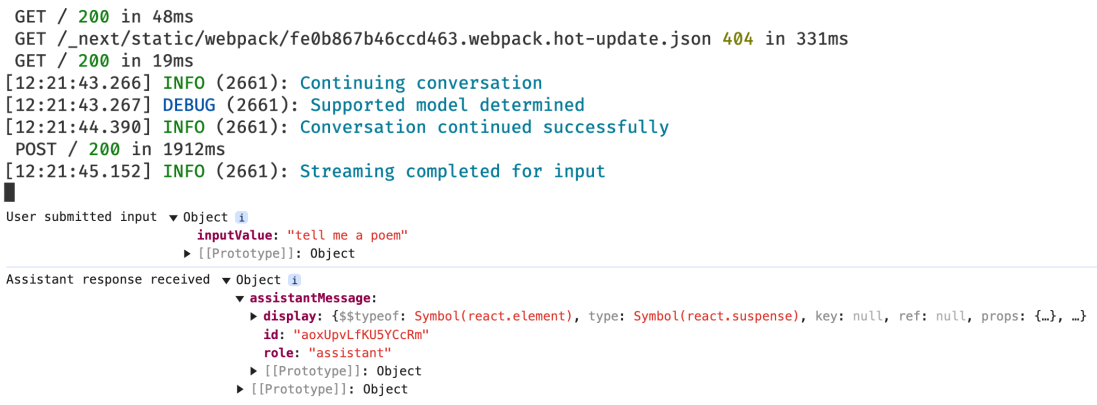


Figure 8.3 Console and server log statements in a structured format help us understand how the client and server communication works.

As we've seen, we can use logging to track state changes and component behavior. This helps identify where and when state mutations occur and can reveal unexpected state transitions.

We've also discussed the importance of maintaining a clear separation of concerns by extracting AI-related logic from UI components. This practice makes the state flow more predictable and easier to debug while also improving code maintainability. For example, in chapter 4, we configured Vercel AI SDK UI and AI state management components to help us achieve this separation effectively.

Ultimately, the primary consideration in state management is understanding where your data resides within the application and how it is being updated. Once you have complete visibility into both aspects, it becomes easier to pinpoint problems that may affect your web application's behavior.

8.1.4 Performance monitoring

Some performance challenges emerge from the complexity of data handling, API interactions, and state management in Next.js applications that utilize generative AI. The asynchronous nature of interactions with AI providers means that requests can take several seconds to complete. This delay can lead to a perception of sluggishness in the application, especially if errors occur behind the scenes or the application retries some request. Since each retry adds to the overall response time, it can significantly affect user experience.

To mitigate performance problems, we need to implement effective performance optimization techniques. Let's look at three key strategies for monitoring performance of your Next.js applications: network monitoring, memory profiling, and the `Performance.now()` API.